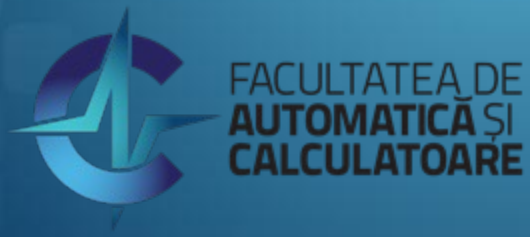


Algoritmi Paraleli și Distribuiți
Alegerea unui lider

Radu-Ioan Ciobanu
radu.ciobanu@upb.ro



Alegerea unui lider

- Gerard LeLann a propus problema alegerii unui lider într-un paper celebru din 1977
- Multe sisteme distribuite sunt bazate pe paradigma client-server:
  - Un server/coordonator (lider)
  - Mai mulți clienți
- Ce se întâmplă dacă liderul se defectează?
  - Soluție: se alege un nou lider
- Problema generică: dintr-o colecție de procese, se alege unul singur care urmează să joace un rol special *(lider)*:
  - Execută operații de inițializare
  - Controlează alte procese

INFORMATION PROCESSING 77, B. GILCHRIST, EDITOR
© IFIP, NORTH-HOLLAND PUBLISHING COMPANY (1977)

DISTRIBUTED SYSTEMS—TOWARDS A FORMAL APPROACH

GÉRARD LE LANN
IRISA—Université de Rennes—BP 25 A
35 031 Rennes Cedex, France

Packet-switching computer communication networks are examples of distributed systems. With the large scale emergence of mini and micro-computers, it is now possible to design special or general purpose distributed systems. However, as new problems have to be solved, new techniques and algorithms must be devised to operate such distributed systems in a satisfactory manner. In this paper, basic characteristics of distributed systems are analysed and fundamental principles and definitions are given. It is shown that distributed systems are not just simple extensions of monolithic systems. Distributed control techniques used in some planned or existing systems are presented. Finally, a formal approach to these problems is illustrated by the study of a mutual exclusion scheme intended for a distributed environment.

1. INTRODUCTION

Computer communication networks using packet-switching technology provide for the interconnection of data-processing equipments of any kind. Such systems, sometimes simply referred to as computer networks, may be viewed as multi-macroprocessors whenever the goals of resource-sharing are achieved. With the large-scale emergence of mini and microcomputers, it is now possible to envision building general or special purpose multimini and multimicrocomputers to be operated in a non-centralized manner. The need for automatic resource-sharing arises here as in a similar way it does for multimacroprocessor systems.

Two kinds of resources must be considered :
- system resources, multi-accessed by users and for which multiplexing is required (hidden sharing)
- user resources, which users agree to share according to some protocol of their own (explicit sharing).

This paper discusses the problems of system resource-sharing in a distributed environment. An example of a user-sharing problem is distributed data-base sharing.

2. DISTRIBUTED SYSTEMS-ELEMENTS FOR A FORMAL APPROACH

Experimental and public packet-switching computer

cesses called logical entities.

Let $F = \{f_i, i \in I\}$ be the set of the operating functions and $E_i = \{e_j^i, j \in J(i)\}$ be the set of entities participating in fuction f_i. At any instant t, it is possible to define $s_t(e_j^i)$ as the instantaneous state of entity e_j^i. It is therefore theoretically possible to define the global state of E_i at instant t as the vector $S_t(E_i) = \{..., s_t(e_j^i), ...$ for all $j \in J(i)\}$.

A system will be said to be f_i-centralized if there exists $k \in J(i)$ such that $S_t(E_i)$ is known to e_k^i.

An example is a system in which $\dim(E_i) = 1$; another example is a system in which entities run the operating function f_i by using a common "system table".

A system will be said to be totally centralized if it is f_i-centralized for any $i \in I$.

In contrast, a system will be said to be f_i-distributed if there does not exist $k \in J(i)$ such that $S_t(E_i)$ is known to e_k^i.

A system will be said to be totally distributed if it is f_i-distributed for any $i \in I$.

Typical cases of distributed systems are systems in



Le Lann, Gérard. "Distributed Systems-Towards a Formal Approach." *IFIP congress*. Vol. 7. 1977.

Alegerea unui lider

- Desemnarea statică a liderului ne-aplicabilă
  - nu se cunoaște compoziția exactă a grupului de procese
- Fiecare proces își cunoaște propria identitate și vecinii
- Identitățile proceselor aparțin unei mulțimi total ordonate
- Alegerea liderului = determinarea procesului care are identitatea cea mai mică *(sau cea mai mare)*
- Alegerea se face prin algoritmi descentralizați, cu participarea tuturor proceselor din colecție

Alegerea unui lider

- Problemă:
  - Atunci când un proces se defectează, mai multe procese pot depista problema simultan
- Nu putem presupune că un singur proces detectează defectul
  - Algoritmul de alegere trebuie să conducă în final la situația în care toate procesele decid asupra aceluiași proces ca nou lider

Alegerea unui lider cu algoritmi undă

Caracteristici:

- toate procesele au același algoritm local
- algoritmul este **descentralizat** *(poate fi inițiat de oricare subset de procese)*
- în fiecare calcul algoritmul atinge o configurație finală în care:
  - un proces este lider
  - toate celelalte au pierdut
- varianta mai slabă:
  - un proces este lider
  - celelalte procese nu știu că au pierdut

Alegerea unui lider cu algoritmi undă (2)

- Fiecare proces are o **stare** dintre:
  - leader
  - lost
  - sleep *(procesul nu a executat nici o acțiune)*
  - candidate *(a executat acțiuni dar nu e sigur dacă e leader sau lost)*

- Ipoteze:
  - sistemul este asincron *(comunicație asincronă, fără timp global, timpi arbitrari de transmisie)*
  - fiecare proces este identificat printr-un nume unic
  - identitățile sunt extrase dintr-o **mulțime total ordonată**
  - aflarea liderului se transformă în **aflarea procesului cu cel mai mic identificator**

Algoritmi undă pentru alegerea liderului

- Algoritmii undă au cele trei proprietăți:
  - Terminare
    - Algoritmul se termină într-un număr finit de pași
  - Decizie
    - Cel puțin un nod execută un eveniment *decide*
  - Consistență
    - *Oricare* proces execută cel puțin un eveniment ce precede cauzal evenimentul *decide*

Folosirea algoritmilor undă pentru alegerea liderului

- Ideea:
  - Fiecare proces are atașată o identitate în cadrul undei
    - Un nod decide atunci când un eveniment a precedat cauzal fiecare eveniment prin care procesele adaugă identitatea respectivă în vectorul de identități cunoscute
    - Procesul ce decide va cunoaște identitatea tuturor proceselor
    - Se alege ca nou lider procesul având identitatea cea mai mică
    - Se anunță *(broadcast)* identitatea liderului tuturor proceselor

Exemplu: alegere lider folosind Algoritmul Arbore

Alegerea unui lider cu algoritmi undă (3)

Alegerea liderului cu algoritm tree

- algoritmul impune ca cel puțin toate frunzele să fie inițiatori ⇒ adaugă o fază de *wakeup:*
  - inițiatorii trimit mesaje *wakeup* tuturor proceselor
  - fiecare proces folosește variabilele:
    - **ws** – boolean, asigură că fiecare proces transmite mesaje *wakeup* cel mult o dată
    - **wr** – int, contorizeaza mesajele de *wakeup* recepționate
- când un proces a primit *wakeup* prin fiecare canal, el începe algoritmul de alegere
- când un proces decide, el află identitatea liderului
- în funcție de ea, procesul trece în starea leader sau lost

Algoritmul tree

```
chan ch[1:n](int id_transm, in min_id);
chan wakeup[1:n]();
process Proc[p = 1 to n]{
    bool ws = false;
     int wr = 0, r = număr_vecini_p;
    bool Vecini[1:n] = vecinii_lui_p;
     bool rec[1:n] = ([n] * false);
    int V = p;
    enum state{sleep, leader, lost};
     state stare = sleep;
     int id;
    /* aici începe faza wake-up */
    if (p este initiator) {
         ws = true;
         for [q = 1 to r st Vecini[q]]
              wakeup[q]();
    }
```

Algoritmul tree (2)

```
while (wr < număr_vecini_p){
     receive wakeup[p]();
     wr = wr + 1;
     if (NOT ws) {
          ws = true;
          for [q = 1 to n st Vecini[q]
                send wakeup[q]();
}    }

/* aici începe algoritmul tree */
while (r > 1){
     receive ch[p](q, id);
     rec[q] = true; r = r - 1;
     V = min(V, id);
}
```

Dacă este primul wakeup primit ⇒ trimit wakeup la toți vecinii

Aștept să primesc un răspuns de la N – 1 vecini și fac un minim între răspunsurile acestora.

N = numărul de vecini

Algoritmul tree (3)

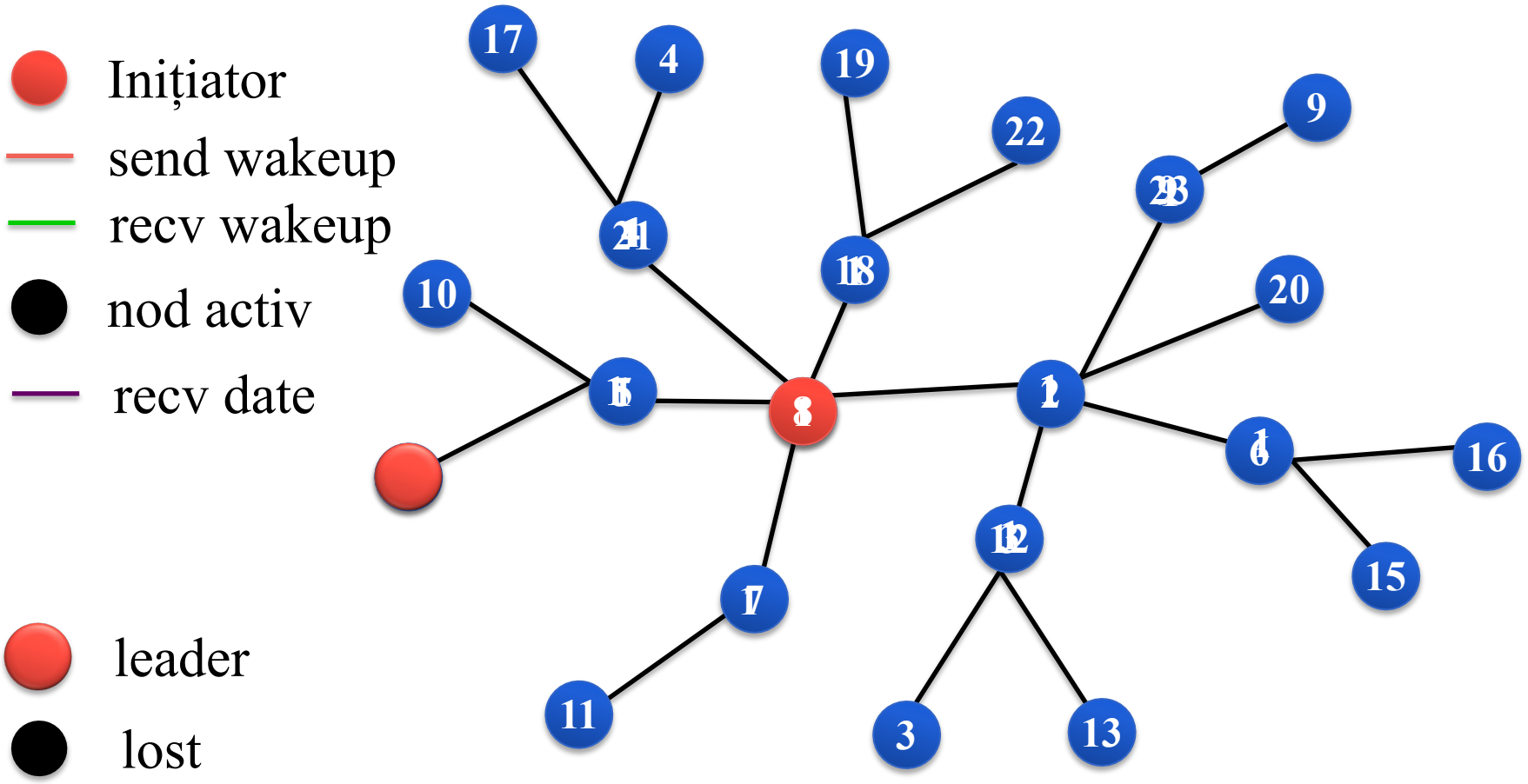
```
/* de la un singur vecin, q0 nu s-a primit mesaj */
afla q0: Vecini[q0] and NOT rec[q0];
send ch[q0](p, V);
receive ch[p](q0, id);
V = min(V, id);
if (V == p)
    stare = leader
else
    stare = lost

/* informeaza celelalte procese despre decizie */
for [q = 1 to n st Vecini[q] and q<>q0]
    send ch[q](p,V);
```

Aștept rezultatul final de la ultimul nod

primit răspuns

Algoritmul tree (4)



Algoritmul tree (5)

- *Notații :*
  - **N** : Numărul de procese implicate
  - **D** : Diametrul rețelei formate de procesele implicate

- *Numărul de mesaje* = O(N)
  - pe fiecare canal se trimit două mesaje *wakeup* și două *token*-uri ⇒ total = 4N - 4

- *Timp* = O(D)
  - în D pași după ce primul proces pornește algoritmul, fiecare proces a trimis mesajele *wakeup* ⇒ în D+1 pași fiecare proces a pornit unda
  - Prima decizie se ia la D pași după pornire, ultima după alți D pași ⇒ totalul = 3D+1 pași

Alegere lider – cazul topologiilor inel

Alegerea liderului - topologie inel

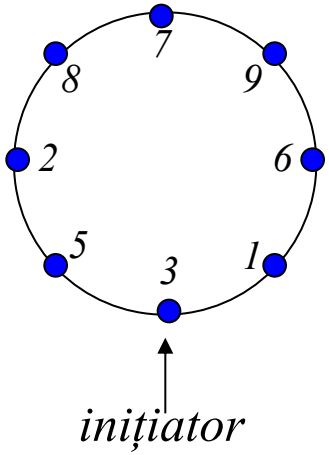
Specificarea problemei:

- Se dă un aranjament circular de procese identificate prin numere distincte între ele
- Dispunerea proceselor în inel este oarecare
- Se cere desemnarea prin conses a procesului cu id **maxim**
- Nu se cunoaște apriori numărul de procese
- Nu există un control centralizat

- Idee simplă:
  - Permitem fiecărui inițiator să trimită un token cu propria identitate pe inel
  - Nodurile nu mai au voie să inițieze după ce au primit un token

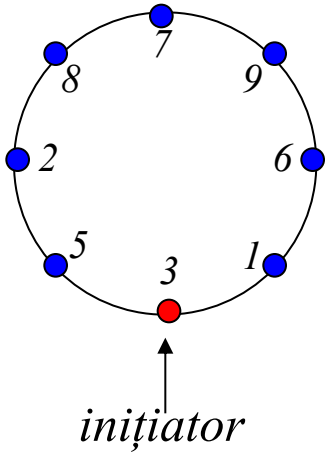
Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...



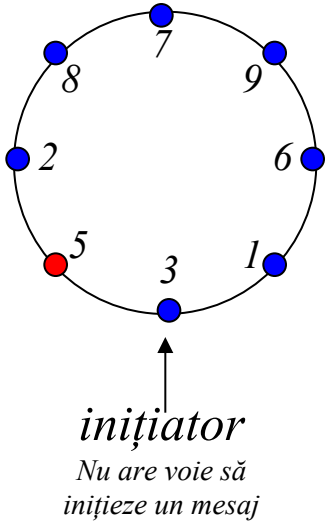
Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...



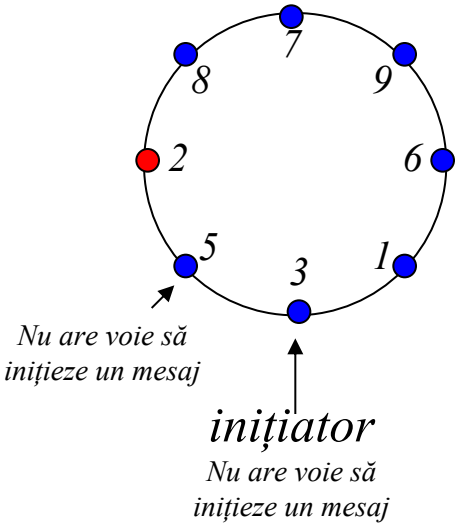
Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...



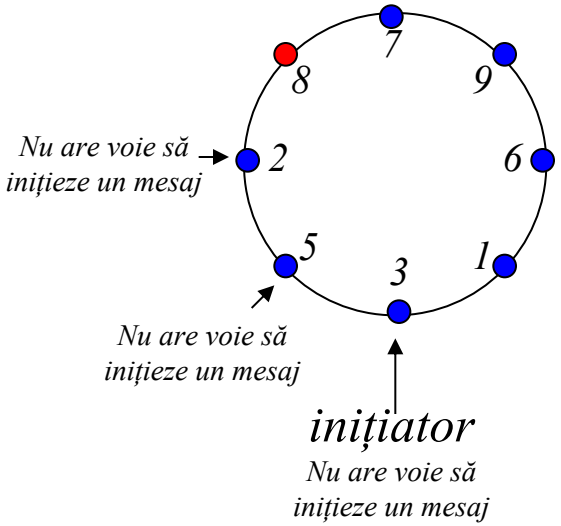
Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...



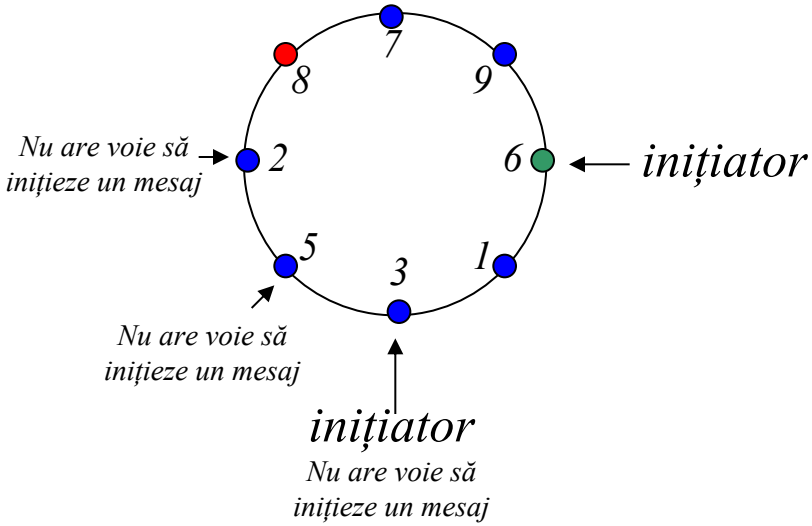
Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...



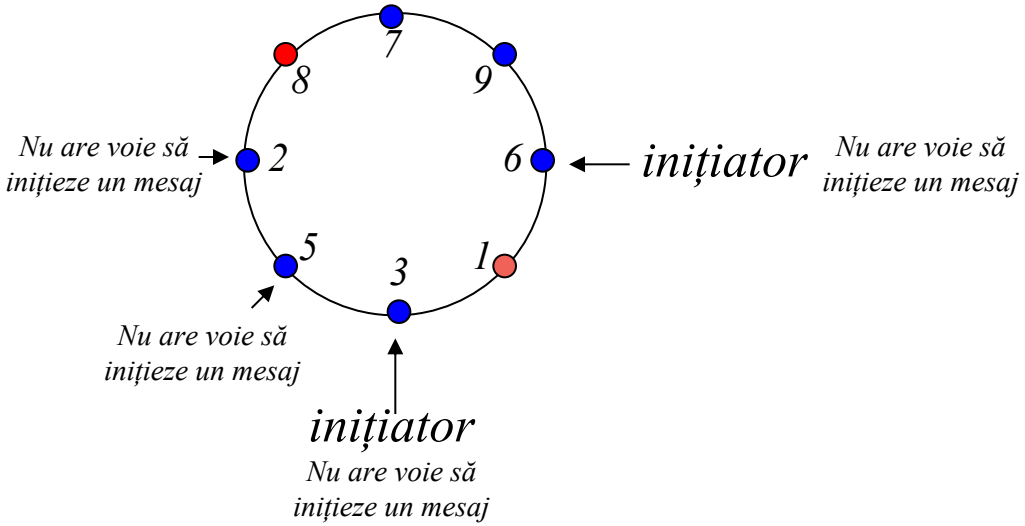
Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...



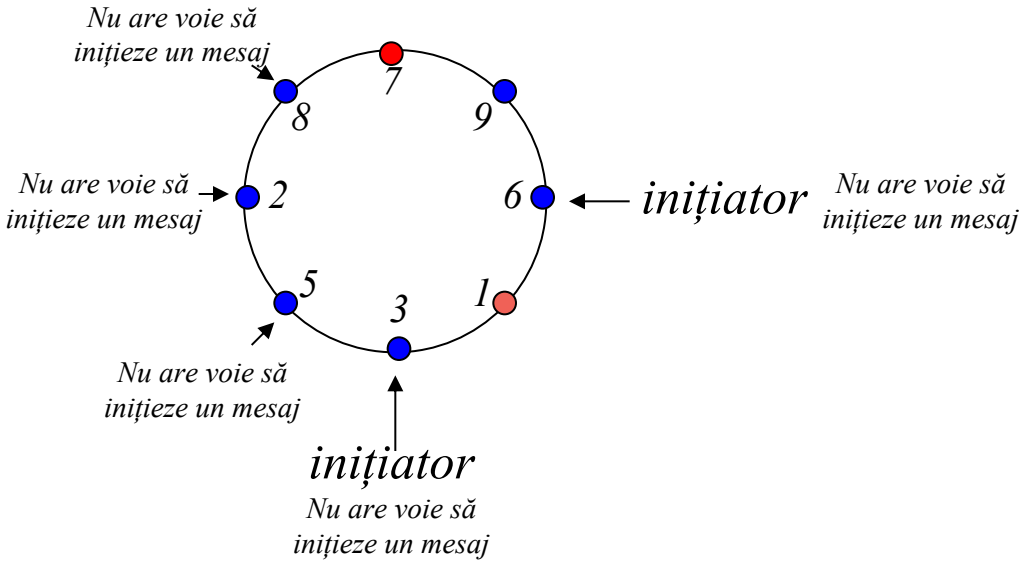
Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...



Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...



Algoritmul LeLann

Algoritmul LeLann

Atunci când un nod primește propriul id, a văzut deja toate celelalte id-uri

- Presupunând canale FIFO

  ⇒ Fiecare proces poate reține o listă cu identificatorii pe care i-a primit...

Algoritmul **LeLann:**

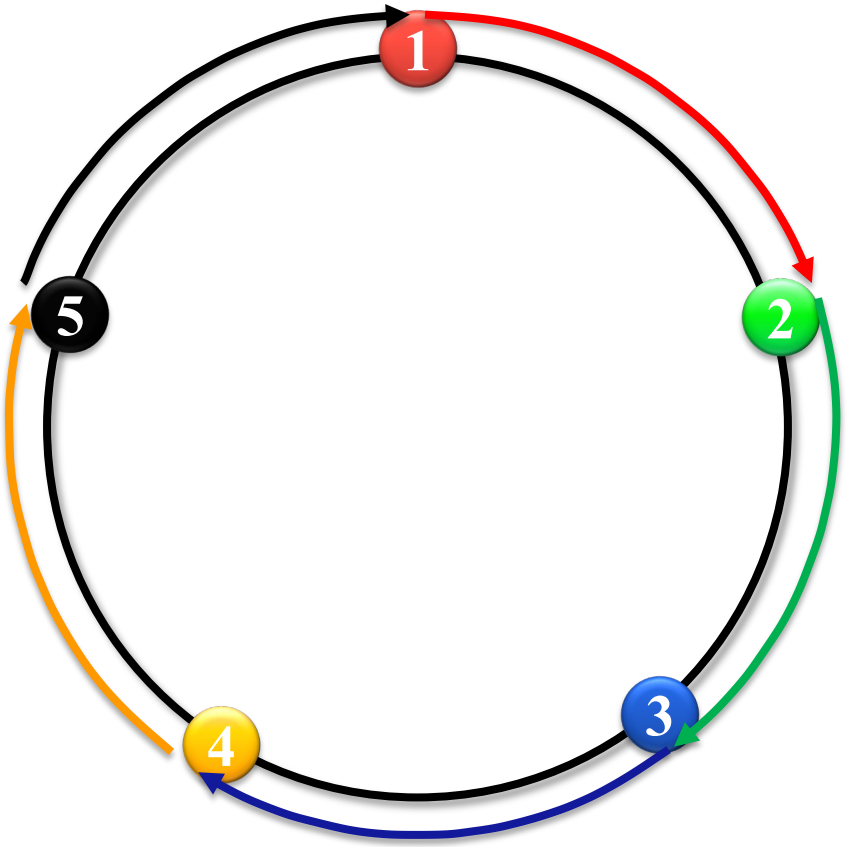
- Fiecare proces difuzează celorlalte un mesaj cu **id**-ul său
- Fiecare proces colectează numerele celorlalte procese
- Fiecare proces află maximul
- Procesul al cărui număr este egal cu maximul devine lider

Algoritmul LeLann

```
chan ch[1:n](token tok, int id_mare);
process Proc[p = 1 to n]{
   typedef SOP{multime identificatori};
   SOP List = {p};
   enum state (candidate, leader, lost);
   state stare = candidate;
   send ch[Urm](tok, p);
   receive ch[p](tok, q);
   while (q <> p) {
       List = List U {q};
       send ch[Urm](tok, q);
       receive ch[p](tok, q);
   }
   if (p == max(List)) stare = leader
   else state = lost
}
```

Număr de mesaje = ? $O(N^2)$

Algoritmul LeLann



Nod	IDs co	Status	
1	{5, 4, 3, 2, 1}	lost 1	5
2	{1, 5, 4, 3, 2}	lost 2	5
3	{2, 1, 5, 4, 3}	lost 3	5
4	{3, 2, 1, 5, 4}	lost 4	5
5	{4, 3, 2, 1, 5}	Leader 5	5

Algoritmul Lelann-Chang-Robert

Operating Systems
R. Stockton Gaines Editor

An Improved Algorithm for Decentralized Extrema-Finding in Circular Configurations of Processes

Ernest Chang
University of Toronto

Rosemary Roberts
University of Waterloo

This note presents an improvement to LeLann's algorithm for finding the largest (or smallest) of a set of uniquely numbered processes arranged in a circle, in which no central controller exists and the number of processes is not known a priori. This decentralized algorithm uses a technique of selective message extinction in order to achieve an average number of message passes of order (n log n) rather than $O(n^2)$.

Key Words and Phrases: decentralized algorithms, distributed systems, operating systems
CR Categories: 4.32, 4.35, 5.25, 5.32

Introduction

Given a random circular arrangement of uniquely numbered processes where no a priori knowledge of the number of processes is known, and no central controller is assumed, we would like a method of designating by consensus a single unique process. The algorithm we propose works equally well for finding either the highest numbered or the lowest numbered process. Let us, without loss of generality, consider highest finding.

A situation in which this algorithm is important has been presented by LeLann [1]. In his example, a circle of controllers in which the control token is lost causes every controller to time out, and an election to find a new emitter for the control token is performed. LeLann's algorithm requires every controller to send a message bearing its number. Each controller thus collects, through the messages seen, the numbers of the other controllers in the circle. Every controller sorts its list, and the controller whose own number is the highest on its list is elected.

LeLann's algorithm, in a circle with n controllers, requires total messages passed proportional to n^2, written $O(n^2)$, where a message pass is a SEND of a message from a controller. This is clearly so, since each of the n controllers sends a message which is passed to all other nodes. Our algorithm requires, on the average, $O(n \log n)$ message passes.

The Algorithm

Each process is assumed to know its own number, and initially it generates a message with its own number, passing it to the left. A process receiving a message compares the number on the message with its own. If its own number is lower, the process passes the message (to its left). If its own number is higher, the process throws the message away, and if equal, it is the highest numbered process in the system.

Proposition: This algorithm detects one and only one highest number.

Argument: By the circular nature of the configuration and the consistent direction of messages, any message



Chang, E., & Roberts, R. (1979). An improved algorithm for decentralized extrema-finding in circular configurations of processes. *Communications of the ACM*, *22*(5), 281-283.

Algoritmul Lelann-Chang-Roberts

- Mesajele sunt transmise în inel **în sensul acelor de ceasornic**.
  - Fiecare proces transmite procesului din dreapta un mesaj cu identificatorul său
  - Un proces care primește un mesaj **m** îl compară cu identificatorul propriu *(id)*:
    - **if** m > id -> transmite m în dreapta
    - **if** m < id -> elimină m
    - **if** m = id -> procesul curent devine lider

- ***Propoziție:***
  - Algoritmul detectează un singur cel mai mare număr

Algoritmul Lelann-Chang-Robert (2)

```
chan ch[1:n](token tok, int id_mare);

process Proc[p = 1 to n] {
    enum state(candidate, leader, lost);
    state stare = candidate;
    send ch[Urm](tok, p);
    while (stare <> lider) {
        receive ch[p](tok, q);
        if (q == p) stare = leader;
        else if (q > p) {
            if (stare == candidate)
                stare = lost;
            send ch[Urm](tok, q);
        }
 }   }
```

Algoritmul Lelann-Chang-Roberts (3)

- Observații:
  - un proces **lost** ramâne în buclă pentru a pasa alte mesaje
  - doar procesul cu id maxim termină
  - pentru a debloca alte procese, liderul poate trimite un mesaj special

- *Algoritm modificat*:

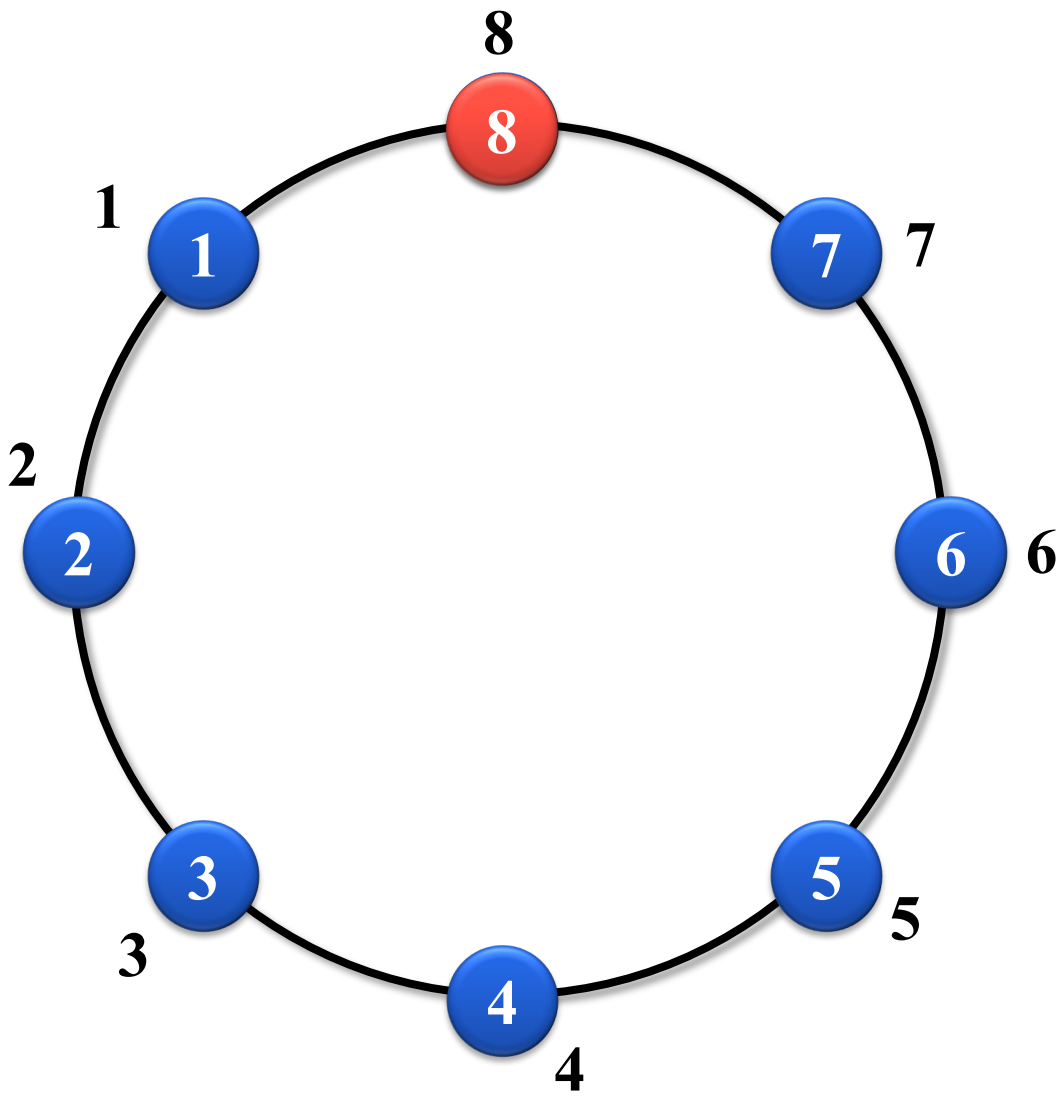
  Când nu toate procesele sunt inițiatori:
  - Cel puțin un proces inițiază alegerea și se auto-marchează
  - Când un mesaj **m** ajunge la un proces nemarcat, acesta:
    - va genera un mesaj cu **id** propriu
    - se va marca
    - va continua cu prelucrarea mesajului **m**

Algoritmul Lelann-Chang-Roberts
Performanță

- Timp
  - Dacă toate procesele pornesc simultan, timpul este $\boldsymbol{O(N)}$, unde $\boldsymbol{N}$ este numărul de procese.
  - Dacă procesul cu numărul maxim pornește primul, timpul este $\boldsymbol{O(N)}$. Altfel, marcajul ia cel mult $\boldsymbol{N-1}$ pași să ajungă la procesul cu id-ul maxim (și apoi $\boldsymbol{N}$ pași pentru propagare), deci timpul este $\boldsymbol{O(N)}$.

- Numărul de mesaje
  - *Cazul cel mai bun.*
    - Procesele sunt ordonate crescător în sensul acelor de ceasornic $\boldsymbol{2N-1}$ mesaje.
  - *Cazul cel mai rău*.
    - Procesele sunt ordonate descrescător în sensul acelor de ceasornic $\sum_{i=1}^{N} i = \frac{N(N+1)}{2}$ mesaje.

Algoritmul Lelann-Chang-Roberts



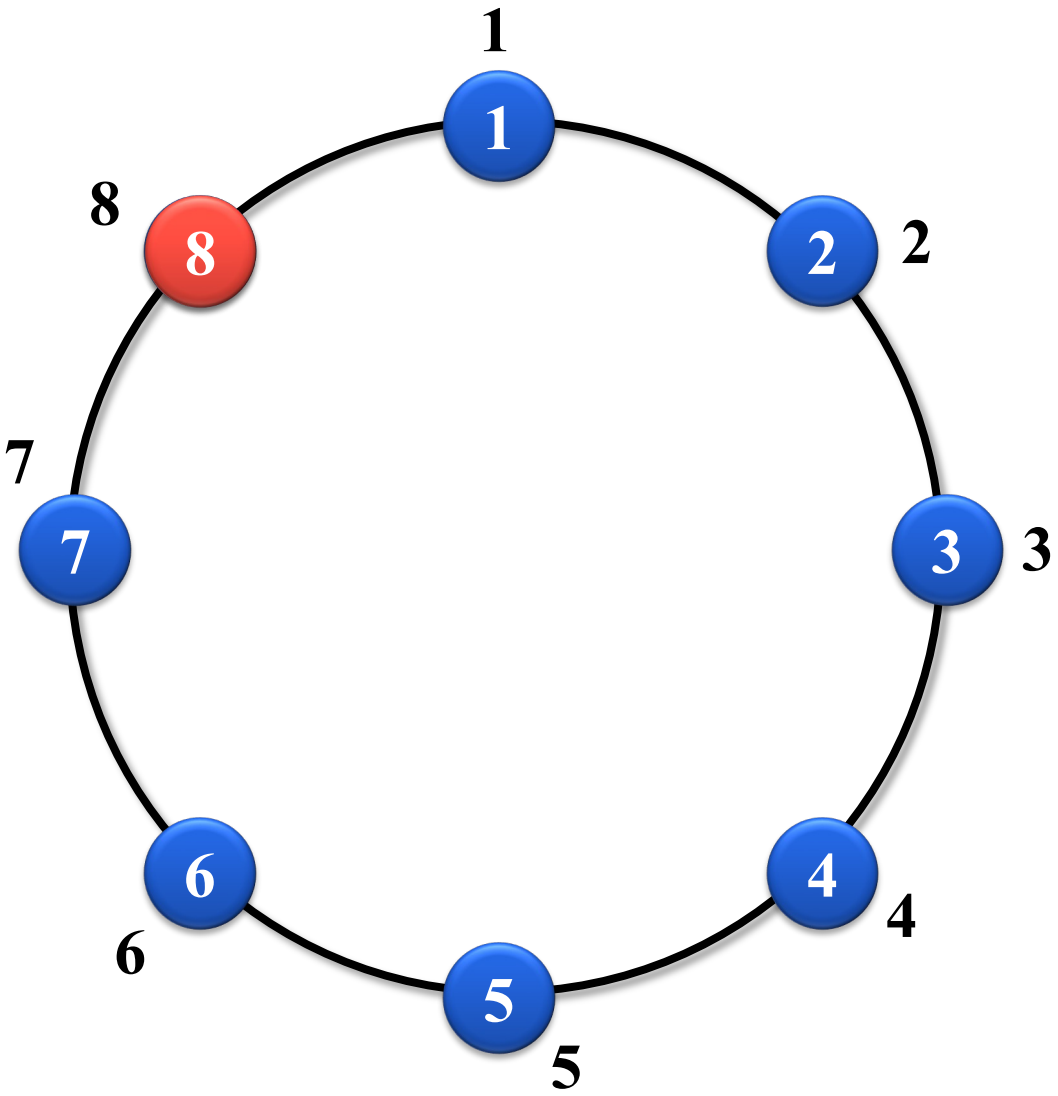
Cazul cel mai rău

candidate

lost

leader

Algoritmul Lelann-Chang-Roberts



Cazul cel mai bun

candidate

lost

leader

Algoritmul Hirschberg-Sinclair

Technical Note
Operating Systems

R. Stockton Gaines
Editor

Decentralized Extrema-Finding in Circular Configurations of Processors

D.S. Hirschberg and J.B. Sinclair
Rice University

This note presents an efficient algorithm, requiring $O(n \log n)$ message passes, for finding the largest (or smallest) of a set of n uniquely numbered processors arranged in a circle, in which no central controller exists and the number of processors is not known a priori.

Key Words and Phrases: decentralized algorithms, distributed system, operating systems
CR Categories: 4.32, 4.35, 5.25, 5.32

Introduction

We are given n processors that are loosely coupled in a circular arrangement and work asynchronously. Each of the processors has an associated unique value (of which it alone is aware) and none of the processors has a priori knowledge of the number of processors in the circle. The problem is to designate by consensus a unique processor from the circle. The total number of data transmissions (messages passed) among the n processors

pass messages in either or both directions, that these directions are distinguished, that processors can detect from which direction a received message originated, but that "left" may not mean the same to all processors. We propose an algorithm that requires $O(n \log n)$ messages in the worst case. The algorithm as given elects the processor with the highest value.

In the algorithm given below, a processor can initiate messages in both directions by a *sendboth* directive. A processor can pass a (possibly modified) message in a circular manner by a *sendpass* directive. A processor can send a responsive message back in the direction from which that processor received a message by a *sendecho* directive.

The Algorithm

```
To run for election:
    status ← "candidate"
    maxnum ← 1
    WHILE status = "candidate" DO
        sendboth ("from", myvalue, 0, maxnum)
        await both replies (but react to other messages)
 IF either reply is "no" THEN status ← "lost"
        maxnum ← 2*maxnum
    OD
On receiving message ("from", value, num, maxnum):
    IF value < myvalue THEN sendecho ("no", value)
    IF value > myvalue THEN DO
        status ← "lost"
        num ← num + 1
        IF num < maxnum THEN sendpass ("from", value, num,
        maxnum)
            ELSE sendecho ("ok", value)
    OD
    IF value = myvalue THEN status ← "won"
On receiving message ("no", value) or ("ok", value)
    IF value ≠ myvalue THEN sendpass the message
        ELSE this is a reply the processor was awaiting
```

The processors initiate messages that are passed in both directions along paths of predetermined lengths (which are successive powers of 2). Processors on the path read the message. If a processor determines, from reading the message, that it cannot win the election, then

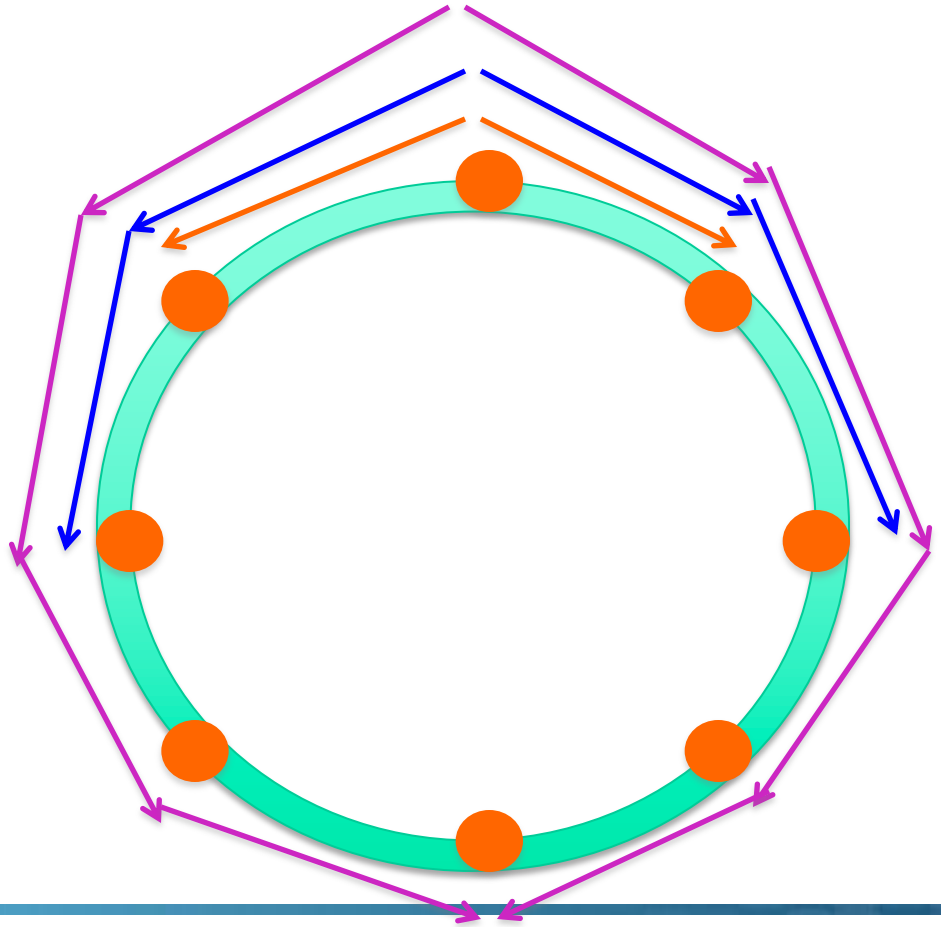


Hirschberg, D. S., & Sinclair, J. B. (1980). Decentralized extrema-finding in circular configurations of processors. *Communications of the ACM*, *23*(11), 627-628.

Algoritmul Hirschberg-Sinclair

- Complexitate $\boldsymbol{O(N\ logN)}$ în cel mai defavorabil caz.
- Lucrează pe un **inel bidirecțional**.
- Procesele pot detecta din ce direcție vine un mesaj și pot trimite un răspuns în acea direcție
- Operații de comunicare folosite de procese:
  - Un proces poate iniția mesaje în ambele direcții prin *sendboth*.
  - Un proces poate pasa un mesaj *(eventual modificat)* prin *sendpass*.
  - Un proces poate trimite un răspuns în direcția din care a primit un mesaj prin *sendecho*.

- Algoritmul opereaza in faze.
- In **faza k**, un proces initiaza mesaje care sunt transmise in ambele directii pe cai de lungime 2^k (1, 2, 4, 8 …)



- Procesele de pe o cale analizeaza mesajul
  - Daca un proces decide ca nu poate castiga alegerea el va pasa mesajul si nu va mai initia mesaje proprii
  - Daca procesul decide ca sursa nu poate castiga alegerea transmite un ecou negativ sursei
  - Ultimul proces din calea 2^k transmite un ecou pozitiv sursei (daca nu poate castiga alegerea)
- Un proces care primeste propriul mesaj castiga
  - in acest caz, calea “acopera” toate procesele
- El va trimite apoi un mesaj pentru a anunta celelalte procese ca a castigat

Algoritmul Hirschberg-Sinclair (2)

Execuția pentru alegere:

```
stare = "candidate"
maxnum = 1
while (stare == "candidate") {
    sendboth ("from", myvalue, 0, maxnum);
    await both replies (but react to other
                        messages);
    if (either reply is "no")
        stare = "lost";
    maxnum = 2 * maxnum;
}
```

Algoritmul Hirschberg-Sinclair (3)

La recepție mesaj ("from", value, num, maxnum):

```
if (value < myvalue)
    sendecho ("no", value)
else if (value > myvalue) {
        stare = "lost";
        num = num + 1;
        if (num < maxnum)
          sendpass ("from", value, num, maxnum)
        else
          sendecho ("ok", value)
    }
else
     stare = "won";
```

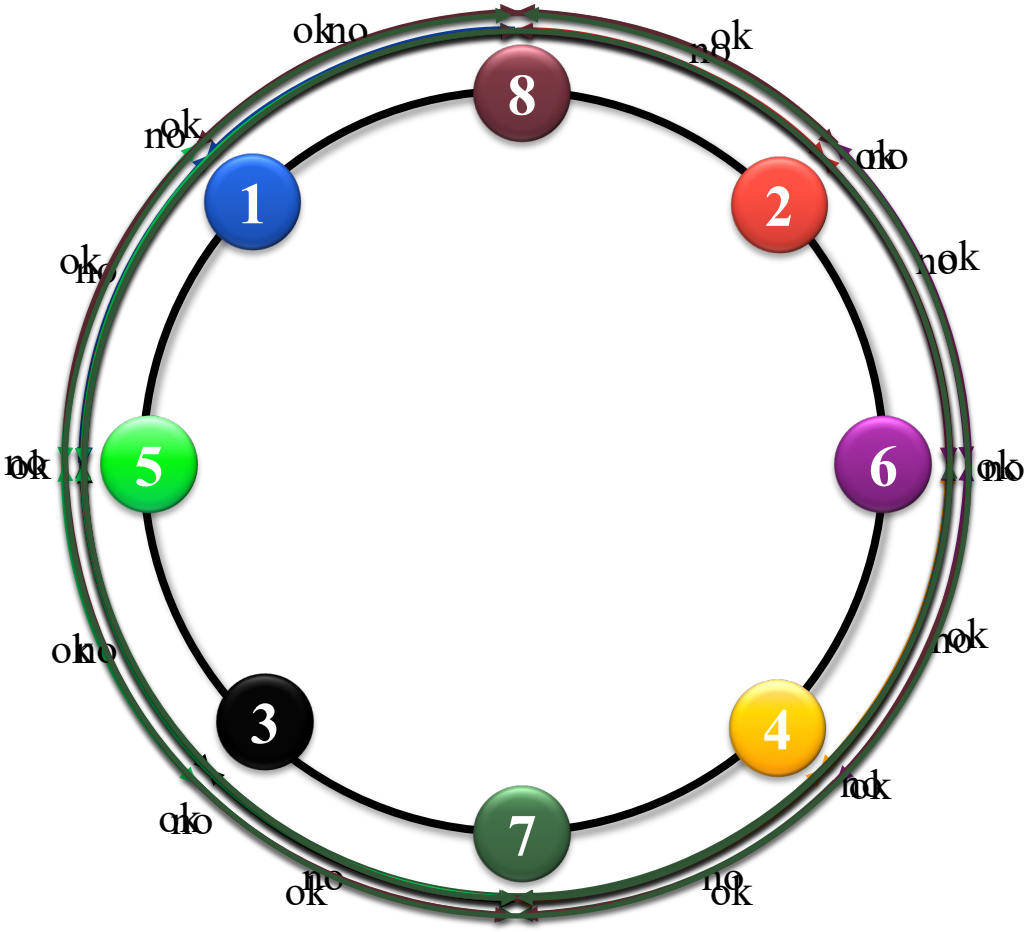
Algoritmul Hirschberg-Sinclair (4)

La recepție mesaj ("no",value) sau ("ok",value):

```
if (value <> myvalue) sendpass the message
else this is a reply the processor was
awaiting
```

- Netratat în algoritm – acțiunile unui proces neinițiator:
  - la primirea unui mesaj devine conștient de alegerea în curs
  - poate deveni *candidate* dacă identitatea are o valoare mai mare decât sursa mesajului

Algoritmul Hirschberg-Sinclair (5)



Complexitate

Numar de mesaje

Un proces initiaza mesaje pe cai de lungime 2^i numai daca el a primit OK in faza pentru distanta 2^{i-1} (in oricare directie)

In orice grup de $2^{i-1} + 1$ procese consecutive, cel mult unul poate initia mesaje pe cai de lungime 2^i →

- toate cele n procese initiaza cai de lungime 1
- cel mult sup[n/2] procese vor initia cai de lungime 2,
- cel mult sup[n/3] de lungime 4,
- cel mult sup[n/5] de lungime 8 etc.

Pentru fiecare cale de lungime 2^i un proces declanseaza cel mult 4 mesaje (2 dus, 2 intors)

Vor fi $4*2^i$ transmiteri declansate de un proces

Complexitate

Numarul total de mesaje este cel mult

$$4*(1*n + 2*[n/2] + 4*[n/3] + 8*[n/5] + \ldots + 2^i*[n/(2^{i-1} + 1)] + \ldots).$$

- Fiecare termen (= lung cale * nr procese) mai mic de 2n
- Sunt cel mult 1+log n termeni
  - in log n faze, calea “acopera” n procese

➔ numar mesaje O(n log n)

Complexitate in timp

Fiecare mesaj este transmis intr-o unitate de timp

Un proces trimite in paralel in cele doua sensuri

Mesaje de la procese diferite se transmit in paralel

$$Timp = 2*(1 + 2 + 4 + 8 + \ldots + 2^i + \ldots + n)$$

Daca n este putere a lui 2 → $Timp = 2*(2^{\log n+1} -1)/(2-1) = 4*n-2$

In general Timp = O(n)

Sumar

- Alegerea unui lider cu algoritmi undă
- Algoritmul tree
- Alegerea liderului – topologie inel
- Algoritmul LeLann
- Algoritmul LeLann-Chang-Robert
- Algoritmul Hirschberg-Sinclair